VIRTUAL CODE

Advanced BACKEND + AI Full Course





System Design



Microservices





In This Course

VIRTUAL CODE

Part 1

Level 0– Basic Backend Revision

Level 1 – Docker
- introduction
- Images & Containers
- How to Dockerize a node Application
- Port Mapping
- Docker Compose
- Docker Networking
- Docker Volumes

Level 2 – Redis
- introduction
- API Caching
- Rate Limiting
- Message Queue

Level 3 – System Design
- introduction
- Scaling Concepts
- Nginx
- Microservices
- Database Scaling (Replication , sharding)

Part 2

Level 4 – AI Full Course
- LLM
- Langchain
- LangGraph
- RAG
- Vector DB

Level 5 – Cloud Deployment
- AWS
- CI/CD Pipeline

Project

“Pay what you think this course deserves” 🤍



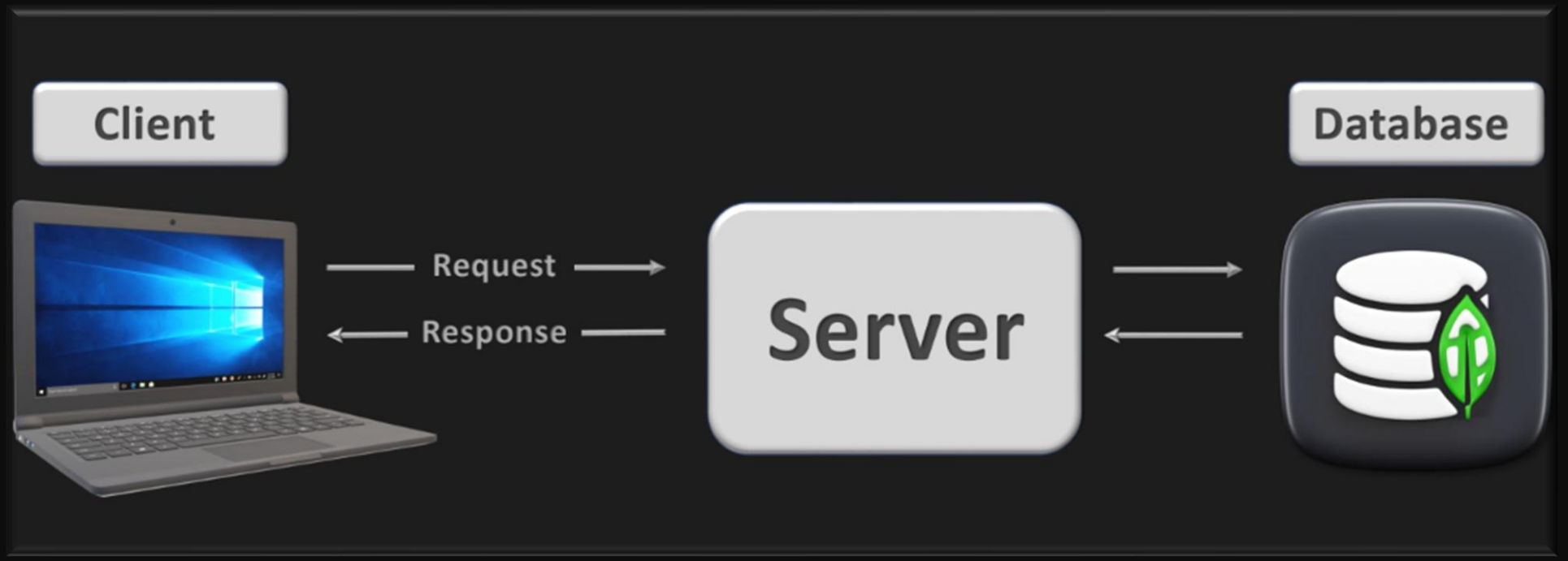
Agar aapko ye course valuable laga ho, to aap apni marzi se support kar sakte ho 🤍
Amount completely aapke upar hai — ₹1 se lekar jitna aapko sahi lage utna.
Bas agar course ne genuinely help kiya ho, to at least ₹50 support kar dena 🙌
Isse aur free & quality content banane me help milegi.

Scan QR or Support via Link in Description

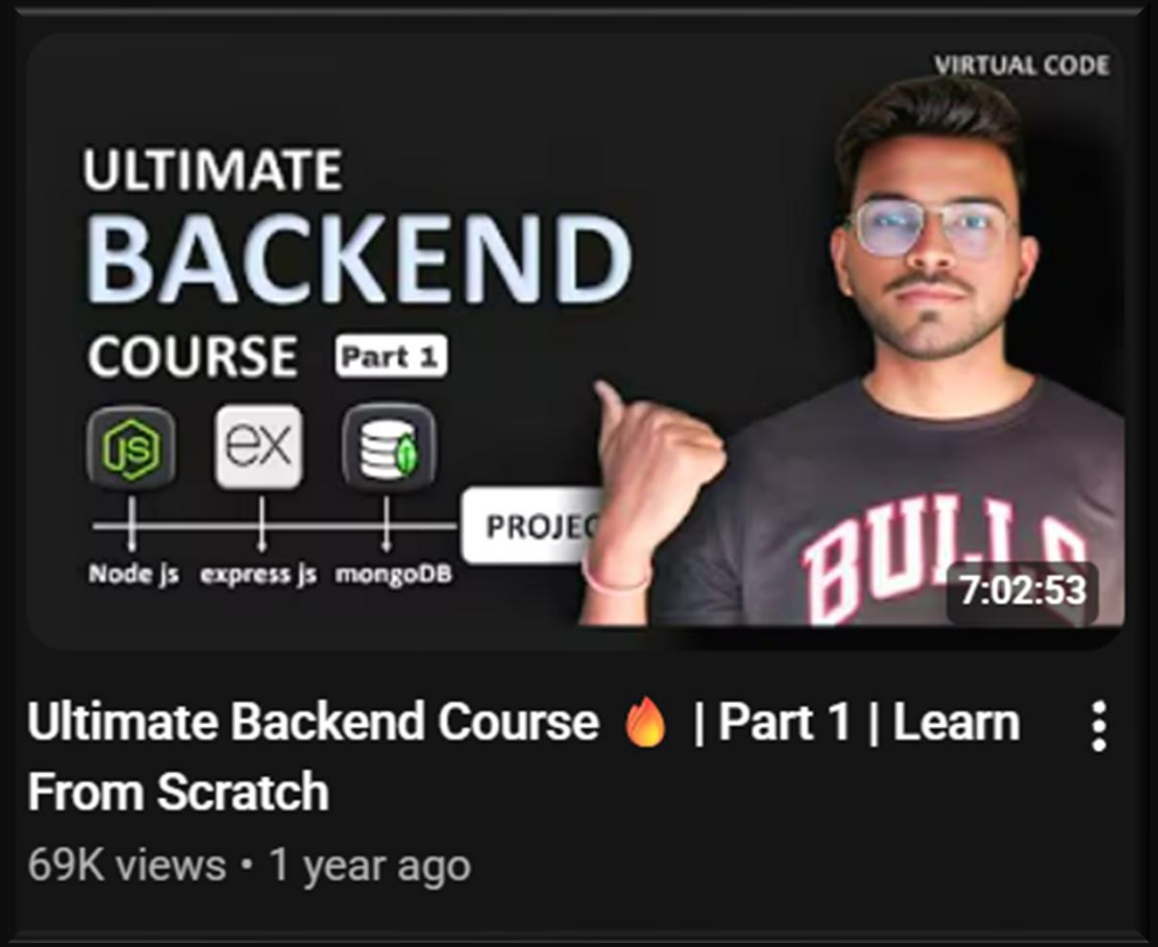
LEVEL 0

Basic Backend Revision

0 | Basic Backend Revision

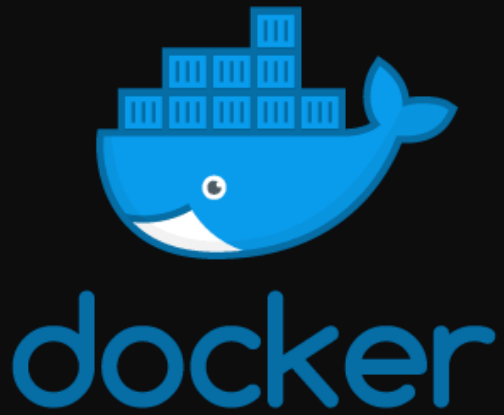


0 | Basic Backend Revision

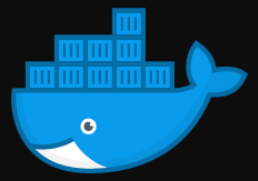


Watch this for Node js , express , mongodb , authentication and more

LEVEL 1



1 | DOCKER

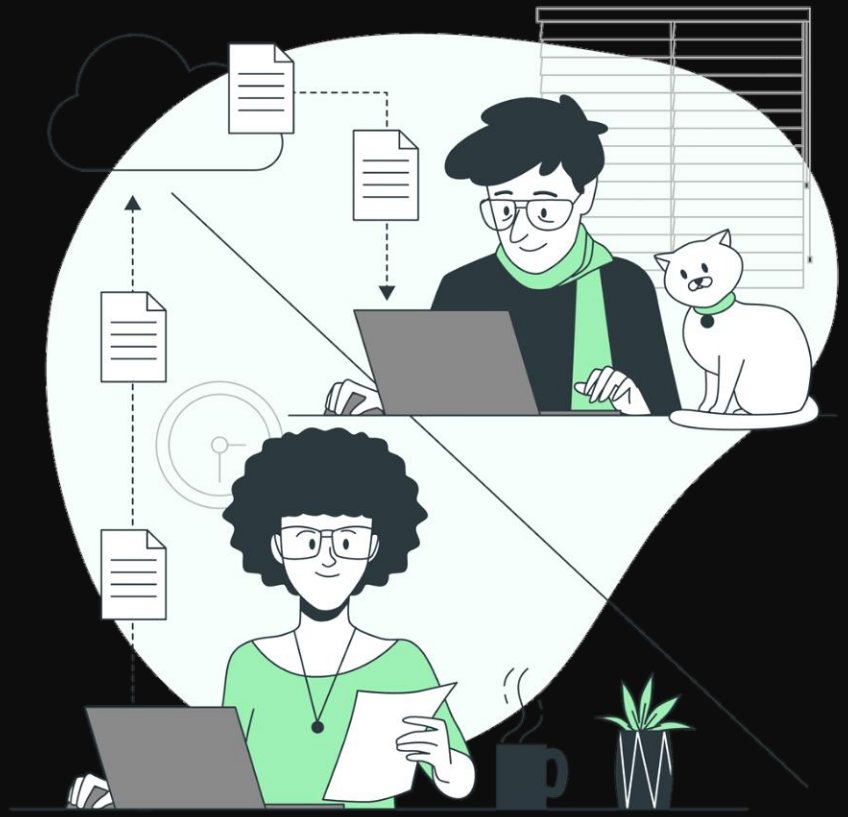


Maan lo tumne ek backend project banaya.

Tumhare laptop me:

- Node.js installed
- MongoDB installed
- sab versions correct

Project perfectly chal raha hai.



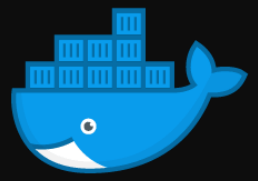




Ab Friend Ko
Project Diya




Mere system me toh chal raha hai
Bhai project chal nahi raha



Why?

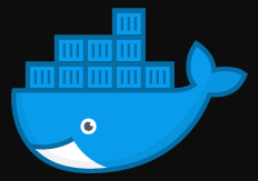
Kyuki uske system me:

- Node version different
- Mongo install nahi
- Environment variables missing

Ye Problem Industry Me Bahut Hoti Thi

Isko bolte hain:

“Works On My Machine Problem”



Docker Ye Problem Solve Karta Hai

Docker application ko ek box/container me pack kar deta hai.

Us box ke andar:

app
node
packages
dependencies
configs
sab hota hai.



Real Life Example — Tiffin Box

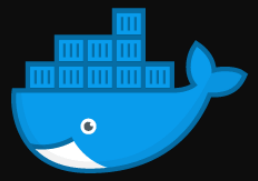
Without Docker

Food alag
Sabzi alag
Roti alag
Spill ho sakta hai

With Docker

Sab ek tiffin box me packed
Safe
Portable
Easy

1 | DOCKER

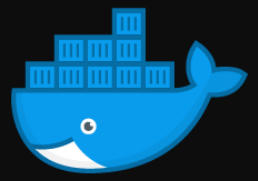


Docker = Shipping Container

Real world shipping containers dekhe honge.
Truck , Ship , Train
Sab same container use karte hain.
Inside kya hai doesn't matter.

Docker Bhi Same Kaam Karta Hai

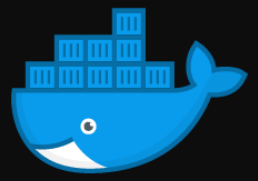
App + Dependencies
↓
Docker Container
↓
Runs Everywhere



Docker Ka Main Goal
“Run Anywhere”
Chahe:

Windows
Linux
Mac
AWS Server

same chalega.



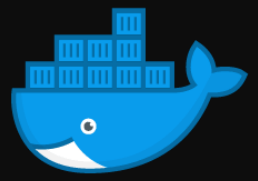
Why Companies Love Docker?

Companies like:

Netflix
Uber
Spotify

use Docker because:

easy deployment
scalable
consistent environments
microservices friendly



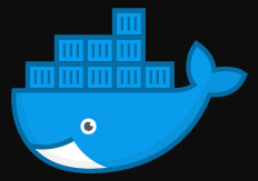
Images and Containers :

Docker Image

Image ek blueprint/template hoti hai.

Image ke andar hota hai:

- app code
- node
- packages
- dependencies
- configs



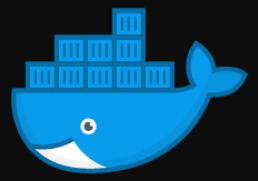
Images and Containers :

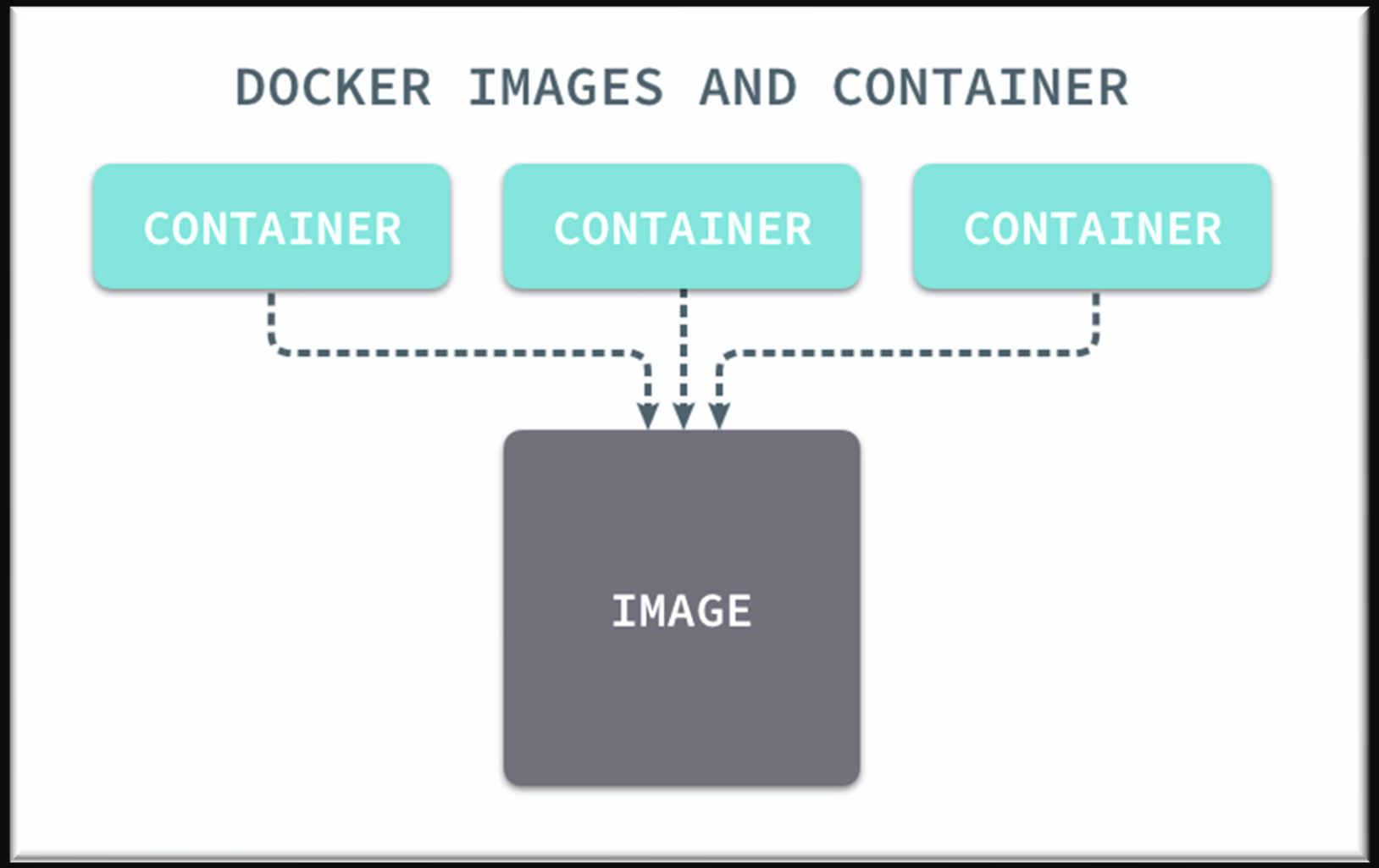
Docker Container

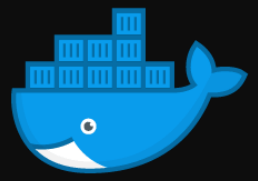
Container image ka running version hota hai.

Container:

- live hota hai
- run karta hai
- processes execute karta hai

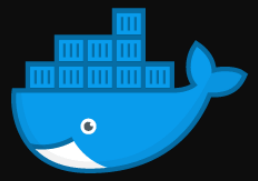



DOCKER IMAGES AND CONTAINER
CONTAINER
CONTAINER
CONTAINER
IMAGE



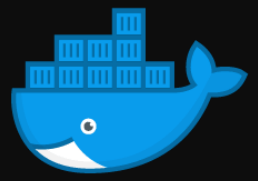
Simple Docker Commands

Command	Work
`docker pull ubuntu`	Ubuntu image download
`docker images`	All images show
`docker run ubuntu`	Ubuntu container run (quickly exit ho jayega)
`docker run -it ubuntu bash`	Ubuntu terminal open
`exit`	Container se bahar aao



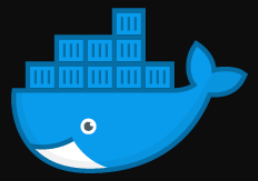
Simple Docker Commands

`docker ps`	Running containers show
`docker ps -a`	All containers show
`docker stop container_id`	Container stop
`docker start container_id`	Stopped container start
`docker rm container_id`	Container delete
`docker exec -it container_id bash`	Running container ke andar jao



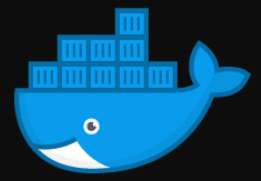
How Docker Works..?

Component	Definition	Work	Example
Docker CLI	Command line interface	User commands receive karta hai	`docker run nginx`
Docker Engine	Main Docker system	Containers aur images manage karta hai	Docker core service
Docker Daemon	Background process/service	Actual Docker operations perform karta hai	Container create/run
Docker Image	Blueprint/template	Application run karne ke liye files/dependencies store karta hai	`nginx`, `ubuntu`
Docker Container	Running instance of image	Actual application run karta hai	Running nginx server
Docker Hub	Online image repository	Images download/upload hoti hain	Docker image store

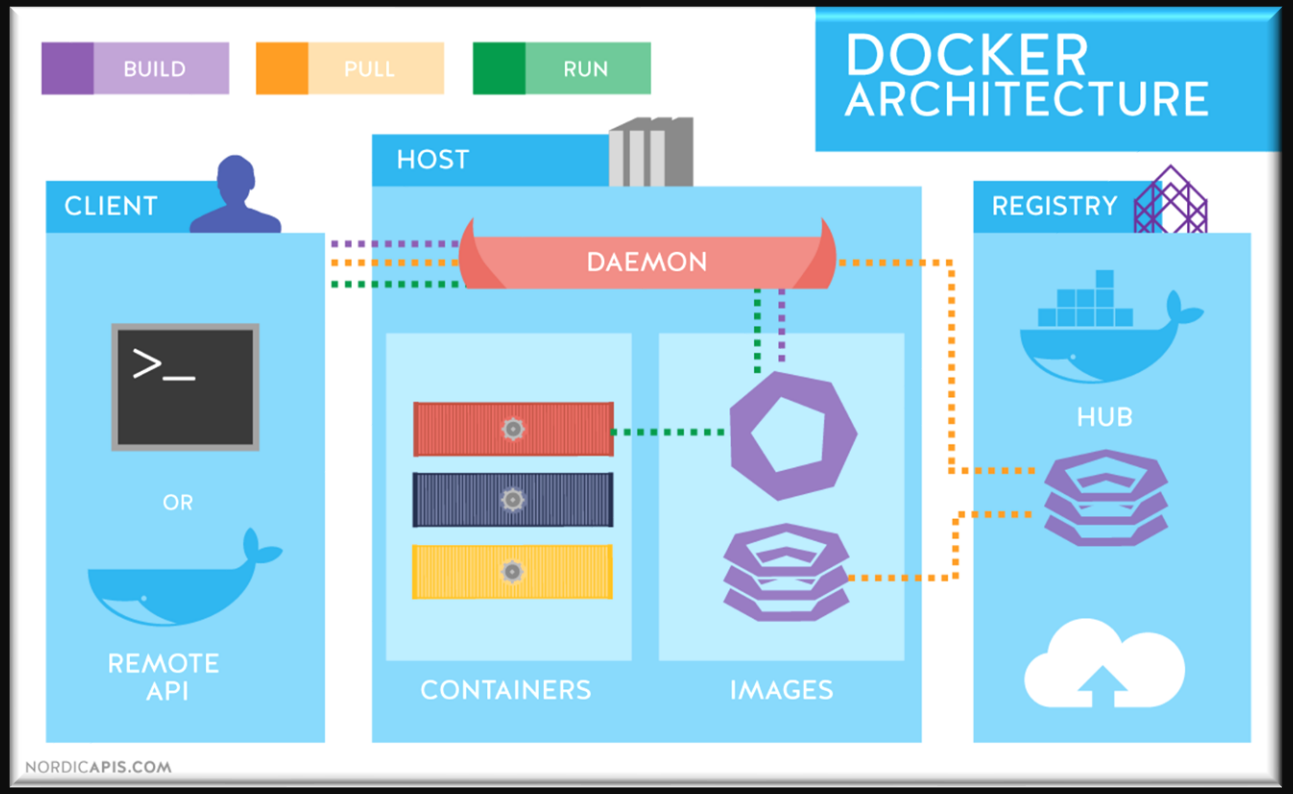


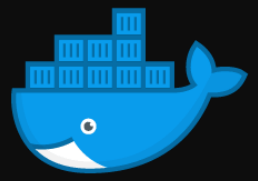
How Docker Works..?

Step	Process
1	User command deta hai
2	Docker CLI command receive karta hai
3	CLI request daemon ko bhejta hai
4	Daemon image check karta hai
5	Missing image Docker Hub se download hoti hai
6	Container create hota hai
7	Application run hoti hai

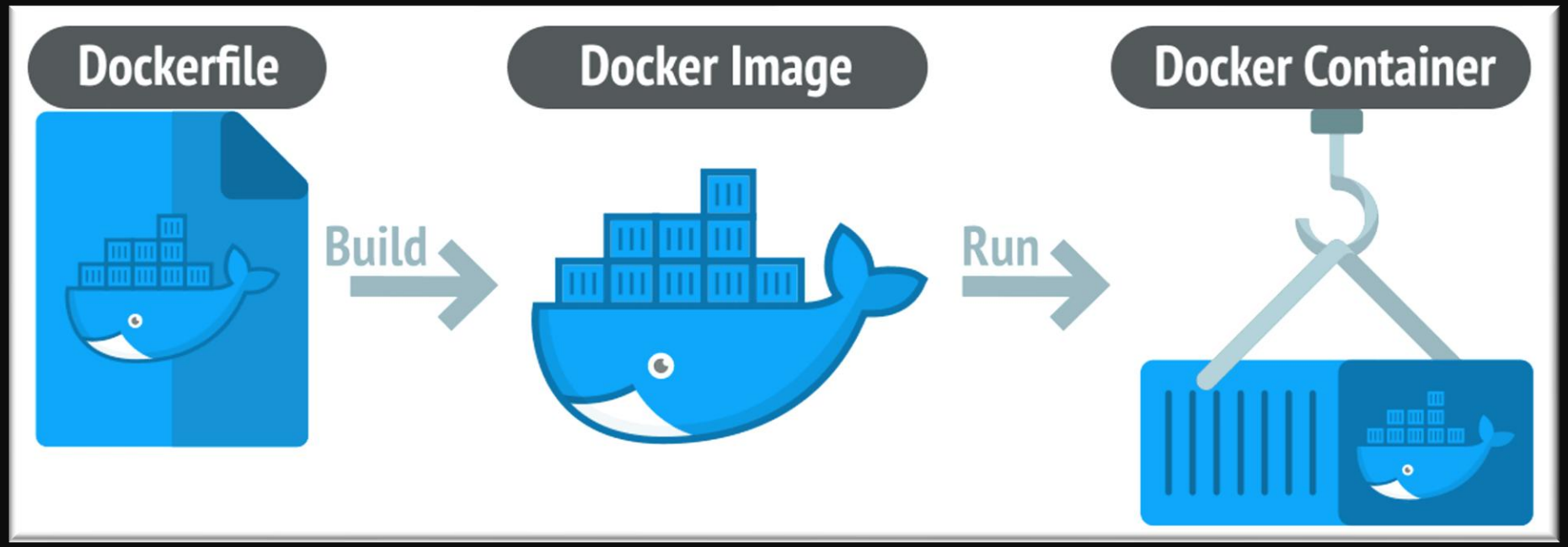


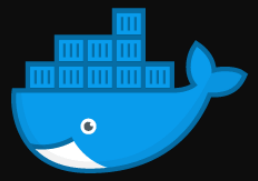
How Docker Works..?





How to Dockerize Node Application -

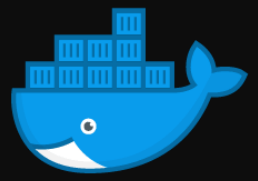




Dockerfile -

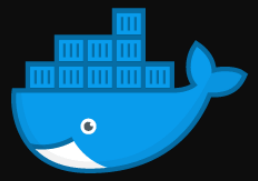
Dockerfile is a text file containing instructions to build a Docker image

`FROM`	Base image choose	`FROM node:20`
`WORKDIR`	Working folder set	`WORKDIR /app`
`COPY`	Files copy	`COPY . .`
`RUN`	Build time command	`RUN npm install`
`CMD`	Container start command	`CMD ["npm","start"]`
`EXPOSE`	App port define	`EXPOSE 5000`
`ENV`	Environment variables	`ENV PORT=5000`
`ARG`	Build variables	`ARG NODE_ENV=production`
`ADD`	Copy + extra features	`ADD file.tar.gz /app`
`ENTRYPOINT`	Fixed startup command	`ENTRYPOINT ["node"]`



Flow of Dockerfile -

```
FROM
 ↓
WORKDIR
 ↓
COPY package.json
 ↓
RUN npm install
 ↓
COPY project
 ↓
ENV variables
 ↓
EXPOSE port
 ↓
CMD start app
```



Dockerfile to Docker Image conversion -

Syntax

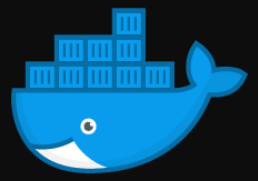
‹/› Bash

```
docker build -t image-name .
```

Example

‹/› Bash

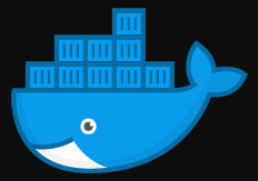
```
docker build -t myapp .
```



Dockerfile to Docker Image conversion -

Meaning

Part	Meaning
`docker build`	Docker image build karo
`-t`	tag/name do
`myapp`	image ka naam
`.`	current folder me Dockerfile hai

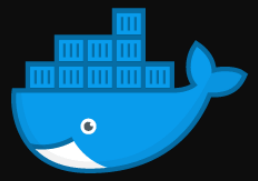


Port Mapping -

Port mapping ka use:

```
container ke andar chal rahi application ko
outside/browser se access karne ke liye
```

hota hai.



Port Mapping -

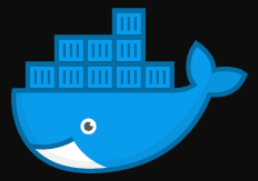
Syntax

Bash

```
docker run -p host_port:container_port image_name
```

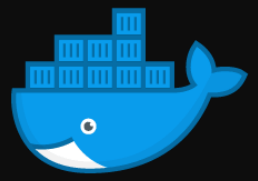
Bash

```
docker run -p 8000:5000 myapp
```



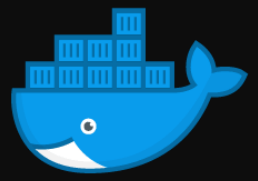
Add Environment Variable in run time -

```bash
docker run -e PORT=8000 -p 8000:8000 myapp
```



How to Push custom Image on Docker Hub:

```bash
docker push username/image-name
```



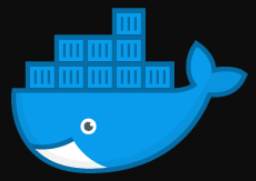
Docker Compose:

Docker Compose is a tool used to run multiple Docker containers together using one file and one command.

Main File

Docker Compose ki file:

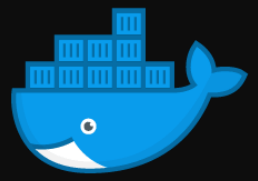
```
docker-compose.yml
```



Docker Compose:

```yaml
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}

  redis:
    image: redis:alpine
```



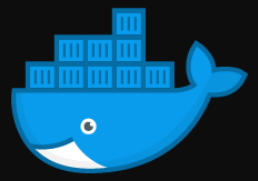
Docker Networking :

Docker network allows containers to communicate with each other.

Check Networks

Bash

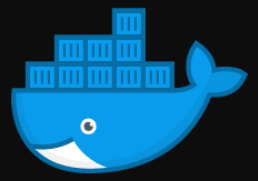
```
docker network ls
```



Docker Networking :

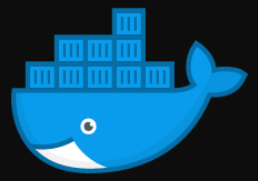
Most Important Network Types

Type	Purpose
bridge	default local network
host	host machine network
none	no network
custom bridge	custom communication



Docker Networking :

`docker network ls`	show networks
`docker network create`	create network
`docker network inspect`	network details
`docker network rm`	remove network
`docker run --network`	attach container



Docker Volumes :

Docker volumes store data permanently outside containers so data remains safe even if containers are deleted or restarted.

Type	Syntax	Data Kahan Save Hota Hai	Use Case	Example Command
Bind Mount	`-v "local-folder:container-folder"`	Local Machine	Development	`docker run -it -v "D:\project:/app" ubuntu`
Named Volume	`-v volume-name:container-folder`	Docker Internal Storage	Database Storage	`docker run -it -v mongo-data:/data/db mongo`

Redis : In memory cache

LEVEL 2





Gada Electronics 📺

Situation

Customers baar baar same cheez puch rahe:

```
iPhone hai?
Samsung TV hai?
AC available hai?
```

2 | Redis



Without Redis

Har baar:

```
Jethalal godown me check karne jaata
```

Flow

```
Customer
   ↓
Jethalal
   ↓
Godown
   ↓
Product Check
   ↓
Customer
```

2 | Redis 

Problem 😵

Agar:

```
1000 customers aa gaye
```

toh:

```
Jethalal thak jayega 😄
```

Aur:

```
bahut slow
```

ho jayega.

2 | Redis



Smart Solution

Jethalal:

```
popular products counter pe ready rakh deta hai
```

Example

Counter par:

- iPhone
- Samsung TV
- AC

already rakhe hain.

DB se phle cache me jaega , if found :Cache Hit

If cache miss, toh db se and also save in redis , so baad m agar chaiye toh redis se le le



Ab Customer Aaya

```
iPhone chahiye
```

↓

Counter pe already hai.

↓

Instant de diya ⚡

No Need

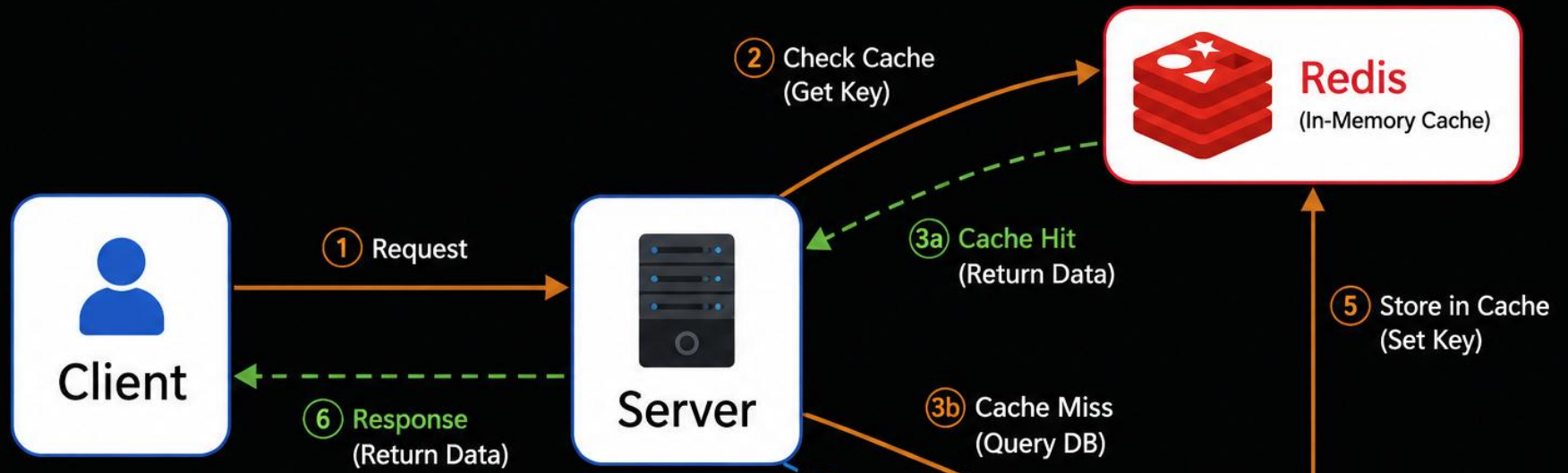
```
har baar godown jaane ki
```



Real Technical Meaning

Real Life	Technical
Godown	MongoDB
Counter	Redis
Jethalal	Backend Server
Customer	User
Product Data	API/Data

2 | Redis



1. Client sends request to Server
2. Server checks Redis for data
3a. If Cache Hit → return data to client
3b. If Cache Miss → fetch data from DB
4. DB returns data to Server
5. Server stores data in Redis (cache)
6. Server sends response to Client

Request / Query Flow

Response Flow (Data)

2 | Redis



Redis Definition

Redis is an in-memory key-value database used for super fast data access.

Use Case	Example
API Caching	products/users
OTP Storage	temporary OTP
Sessions	login
Rate Limiting	API protection
Queues	emails/jobs
AI Memory	chatbot history



API Caching:

Real Flow

```
Request
   ↓
Check Redis
   ↓
Found? Return Fast ⚡
   ↓
Else MongoDB
```

2 | Redis



OTP Storage:

Flow

```
Generate OTP
      ↓
Save in Redis
      ↓
Auto Delete
```

2 | Redis



Sessions:

Login Flow

```
Login
   ↓
Generate JWT
   ↓
Store Session In Redis
   ↓
Cookie Send
```

```
const token = jwt.sign(
  { userId: user.id },
  secret
)

await redis.set(
  `session:${user.id}`,
  token,
  "EX",
  86400
)

res.cookie("token", token)
```



Rate Limiting :

```
ek time me limited requests allow karo
```

2 | Redis 

Attack	Meaning
Brute Force	password guess
Spam	API spam
DDoS	server overload
Bot Abuse	fake traffic

Result 😵

```
server slow/crash
```

Solution 🔥

Rate Limiting

2 | Redis



Real Flow

```
Request
   ↓
Count Requests
   ↓
Limit Cross?
   ↓
Block
```

Redis Commands Used

Command	Work
INCR	count increase
EXPIRE	auto reset

2 | Redis



Suppose IP:

```
1.2.3.4
```

request bhej raha.

Redis:

```
1.2.3.4 → 1
```

↓

next request:

```
1.2.3.4 → 2
```

↓

next:

```
1.2.3.4 → 3
```

After Limit

BLOCK 🚫

2 | Redis



VIRTUAL CODE

```
const rateLimiter = async (req,res,next) => {

    const ip = req.ip
  const key = `rate_limit:${ip}`
    const requests = await redis.incr(key)

    if(requests === 1){
        await redis.expire(key,60)
    }

    const ttl = await redis.ttl(key)


    if(requests > 5){
        return res.status(429).json({
            message:"Too Many Requests",
            retryAfter: ttl
        })
    }

    next()
}
```



Queues:

Queue ek waiting line hoti hai jisme backend tasks store hote hain taaki unhe background me process kiya ja sake bina user ko wait karaye.

Suppose:

```
signup API
```

hai.

Signup ke baad:

- welcome email
- notification
- analytics

karna hai.

2 | Redis



VIRTUAL CODE

WITHOUT QUEUE 😵

JavaScript

```
app.post("/signup", async (req, res) => {

  await sendEmail()

  res.json({
    message: "Signup Success"
  })

})
```

Problem

Agar:

```
email send = 5 sec
```

↓

API bhi:

```
5 sec baad response
```

degi.

User Experience 😵

```
loading...
loading...
loading...
```

2 | Redis



VIRTUAL CODE

WITH QUEUE 🔥

JavaScript

```
app.post("/signup", async (req, res) => {

  await emailQueue.add(
    "send-email",
    {
      email: "ankush@gmail.com"
    }
  )

  res.json({
    message: "Signup Success"
  })

})
```

```
email abhi send nahi hua
```

Bas:

```
email task queue me gaya
```

API Immediately Response

```
Signup Success
```

Then Worker

Background me:

```
email bhejega
```



BullMQ:

BullMQ ek Redis-based queue management library hai jo background jobs handle karti hai.

Component	Work
Queue	jobs store
Job	task
Worker	jobs process



BullMQ:

```
npm install bullmq
```

Folder Structure

```
project/

├── queue.js
├── worker.js

└── Index.js
```

2 | Redis



queue.js:

```
import { Queue } from "bullmq";
import Redis from "ioredis";

export const connection=new Redis("redis://localhost:6379",  {
    maxRetriesPerRequest: null
})

const emailQueue=new Queue("emailQueue",{connection})
export default emailQueue
```

2 | Redis



worker.js:

```
import { Worker } from "bullmq";
import Redis from "ioredis";
import sendMail from "./sendEmail.js";
import { connection } from "./queue.js";

const worker=new Worker("emailQueue",async (job)=>{
 console.log(
        "Job Started"
      )

 const email = job.data.email

  await sendMail(email)

      console.log( "Job Completed" )

}
,
{connection}
)
```

2 | Redis



Add job in queue :

```
await emailQueue.add("send-email",{email})
```

LEVEL 3

System Design

1. System Design:

System Design is the process of designing scalable, reliable, fast, and production-ready software architectures.

Technology	Purpose
Node.js	backend
Redis	cache
Docker	containers
Nginx	load balancer
MongoDB	database

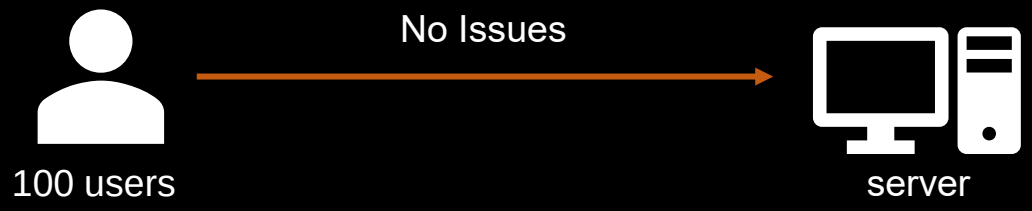
In Sabko Combine Karke 😄

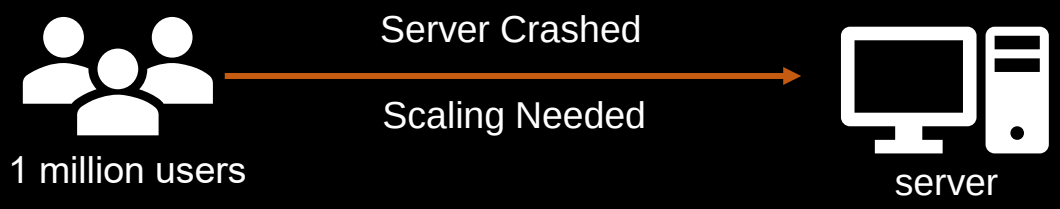
Production architecture banana:

System Design

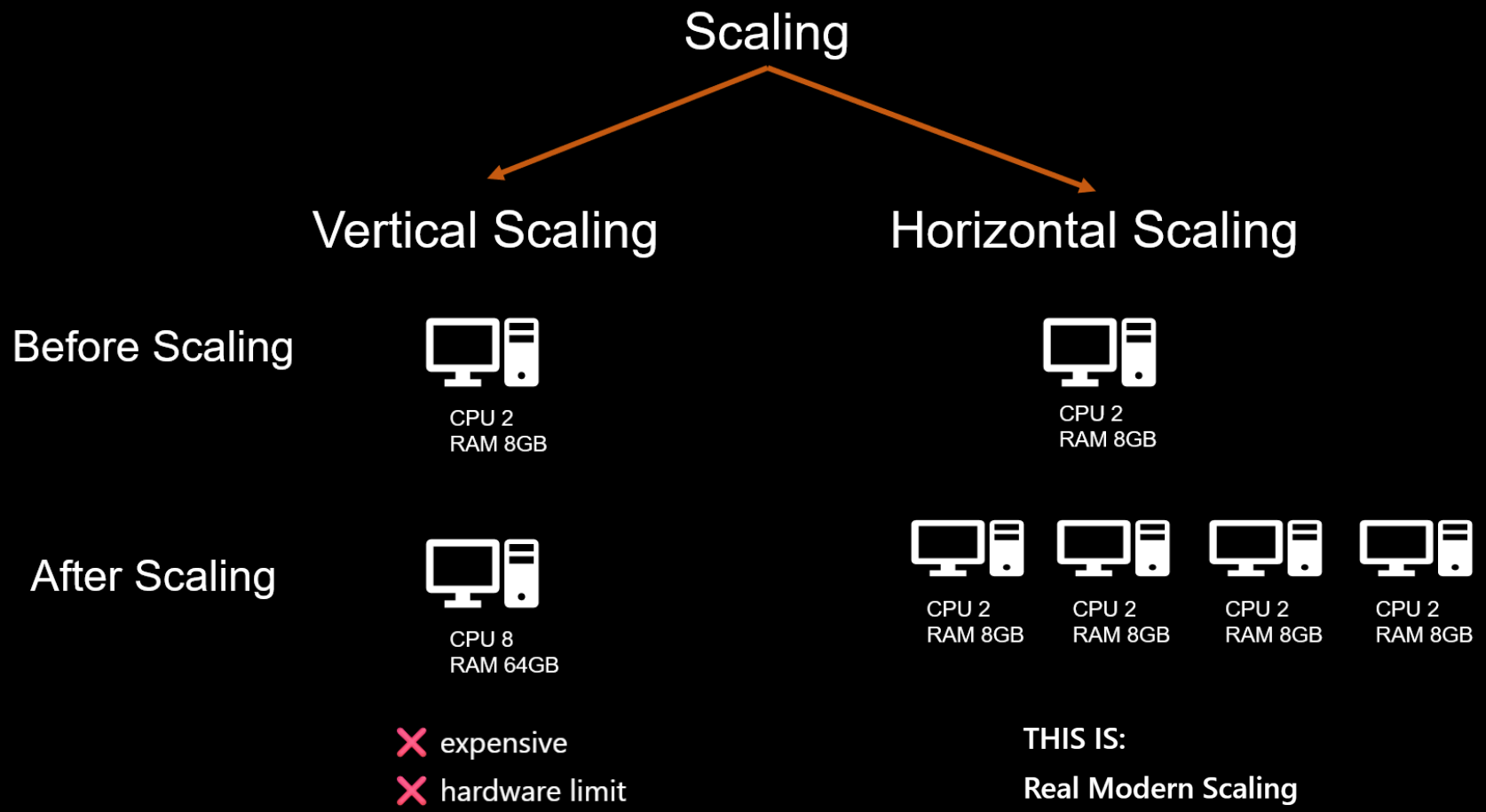
Scaling Concepts :

Scaling means increasing system capacity to handle more users and traffic.

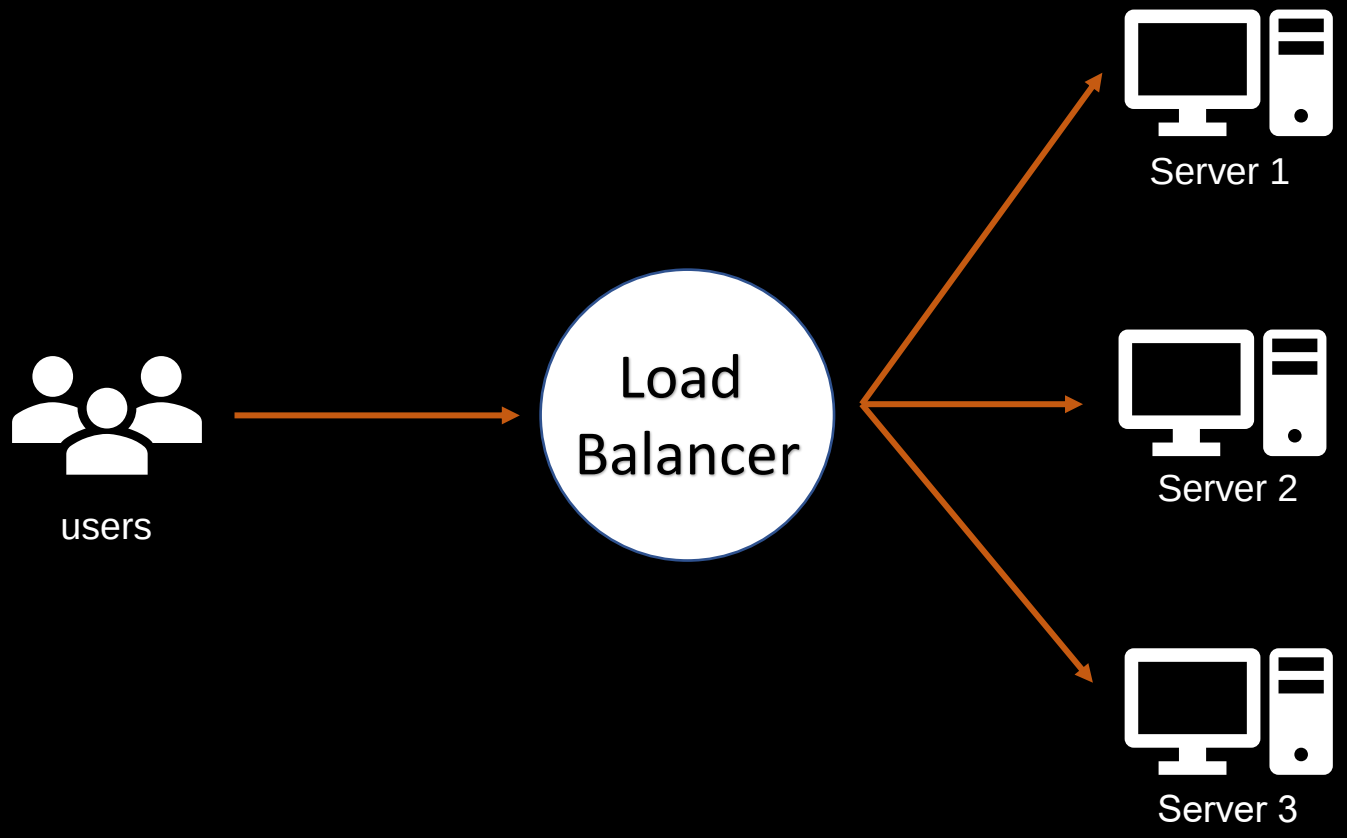




3 | System design



3 | System design



Load Balancer :

Load balancer distributes incoming traffic across multiple backend servers.

Example 🔥

```
Request 1 → Server1
Request 2 → Server2
Request 3 → Server3
```

Load Balancer :

Why Needed?

One server:

```
overload
```

na ho.

Real Load Balancers 🔥

Technology	Type
Nginx	software
AWS ELB	cloud service

Nginx :

Nginx is a reverse proxy and load balancer used to manage traffic between users and backend servers.

Diagram 🔥

```
Users
  ↓
Nginx
  ↓
Backend1
Backend2
```

NGINX FEATURES 🔥

- ✅ reverse proxy
- ✅ load balancing
- ✅ SSL
- ✅ routing

---

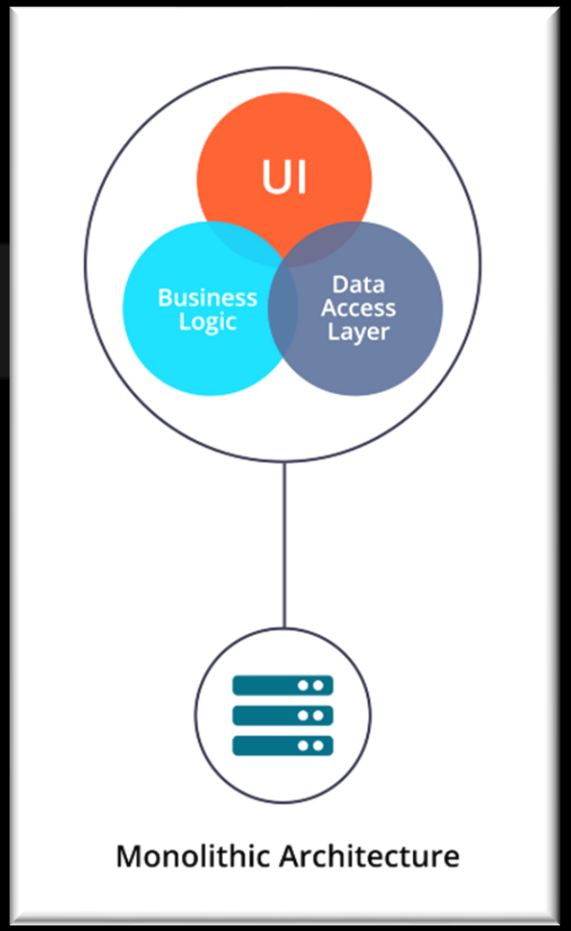
2. Monolith and Microservices :

Monolith

Ek single large application architecture.

```
Frontend → One Backend → One Database
```

Sab modules ek hi project me.



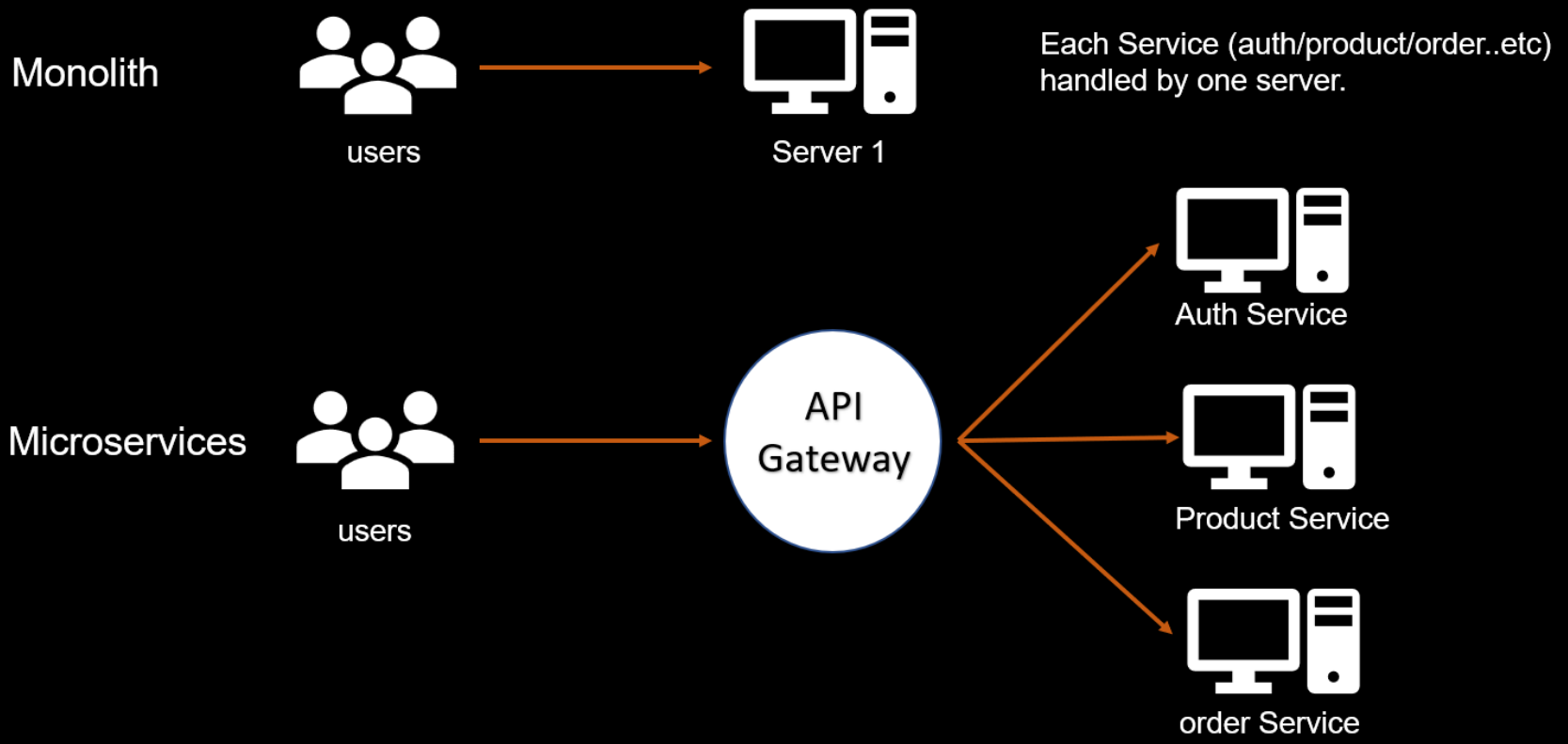
Monolith and Microservices :

Microservices

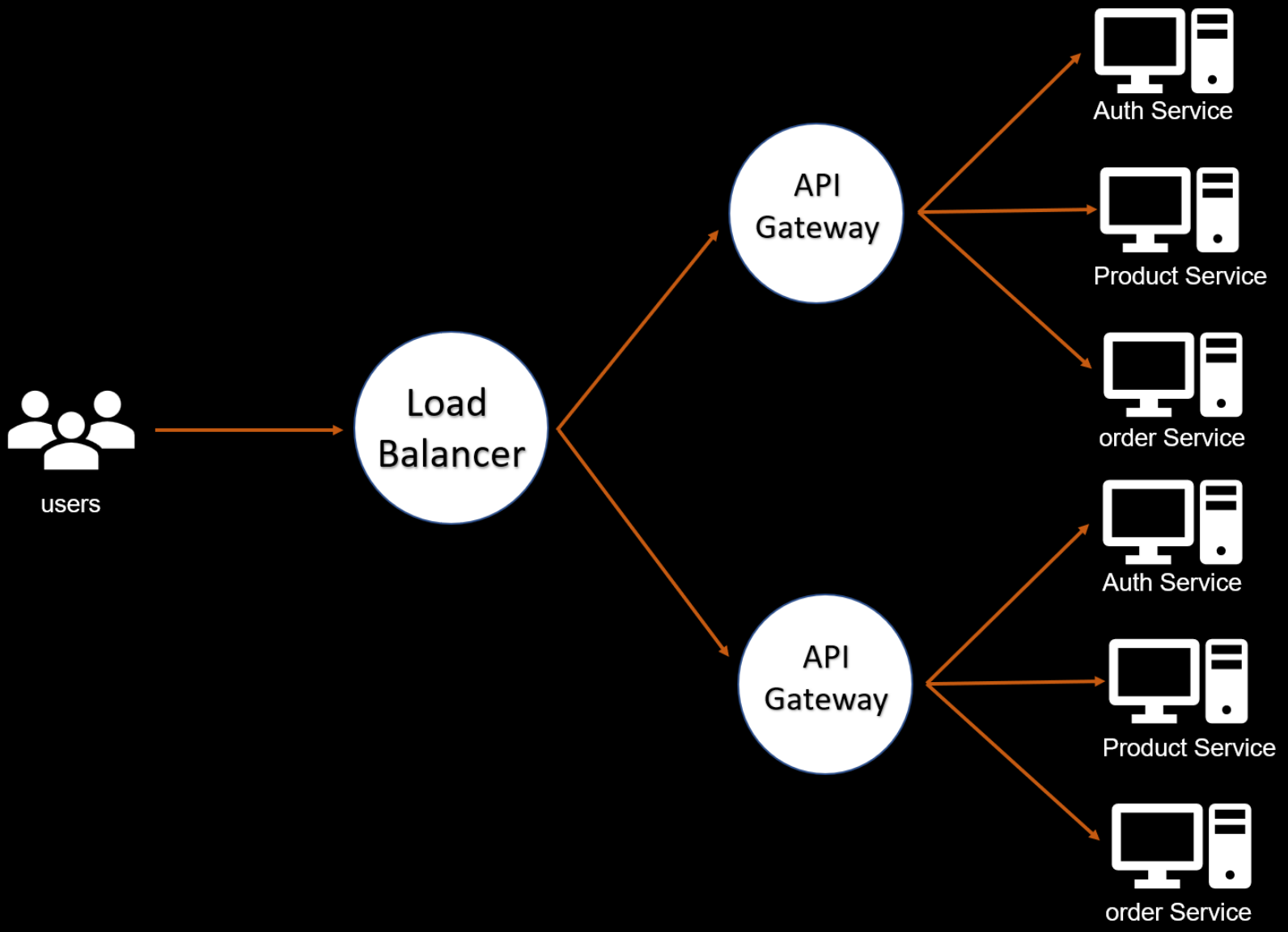
Application ko small independent services me divide karna.

```
Frontend
   ↓
API Gateway
   ↓
Auth Service
Product Service
Order Service
```

3 System design



3 System design



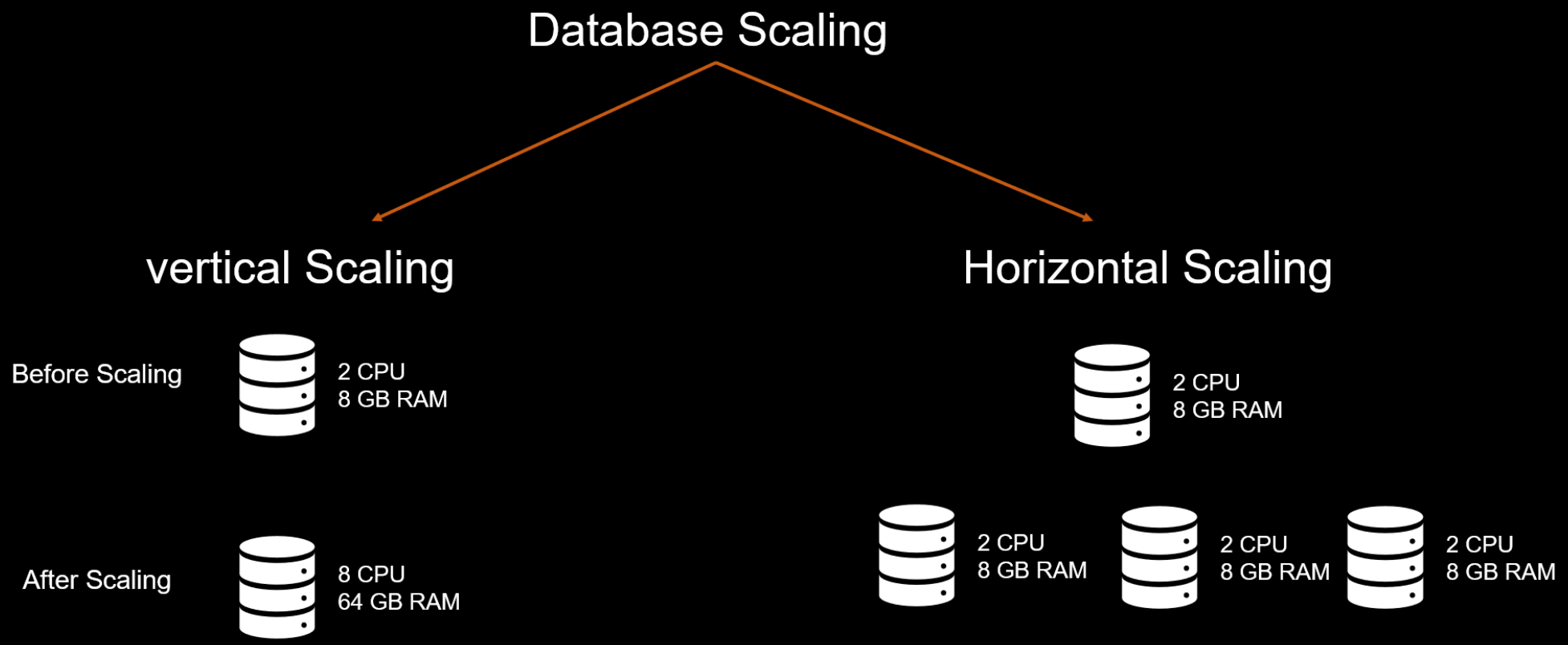
3. Database Scaling:

Definition

Database Scaling ka matlab:

> Database ko itna powerful banana ki wo zyada users, zyada requests aur zyada data handle kar sake.

3 System design



Horizontal Scaling ke 2 Main Concepts

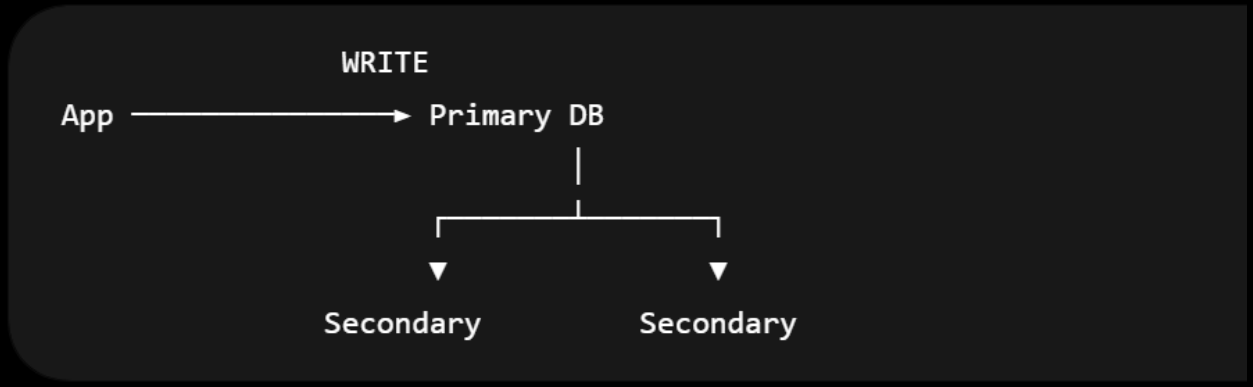
1. Replication
2. Sharding

Replication

Replication me:

> Same data multiple DB servers pe copy hota hai.

Architecture



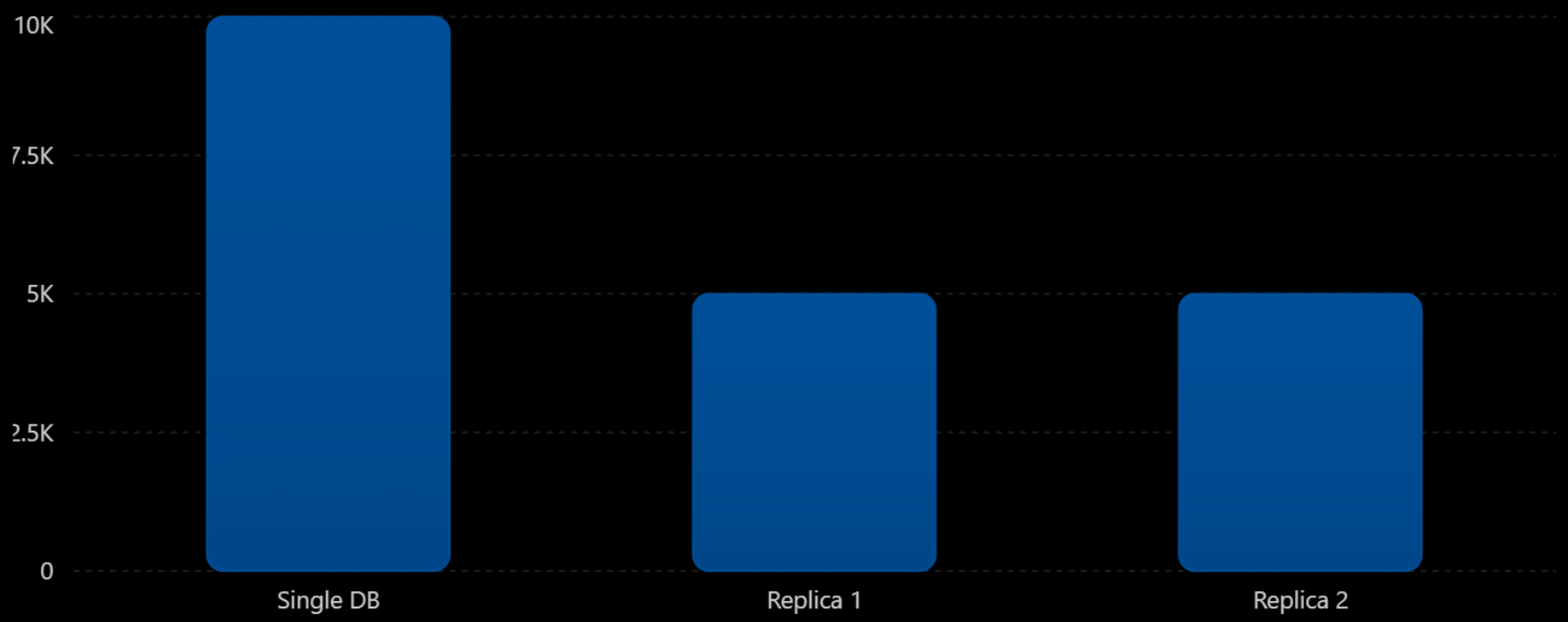
Important Roles

Server	Work
Primary	Writes
Secondary	Reads + backup

Read Scaling

Read load distribution with replication

Replication distributes read traffic across multiple replica nodes.



Sharding

Ab maan lo:

```
Data itself bohot huge ho gaya
```

Single DB storage bhi handle nahi kar paa raha.

Tab use hota hai:

Sharding

Sharding Meaning

> Data ko divide karna multiple databases me.

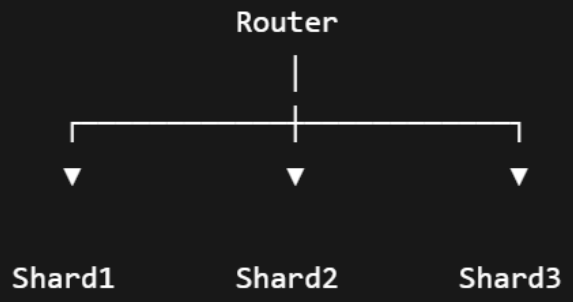
3 System design

Example

Users split:

```
Users 1-1M     → Shard 1
Users 1M-2M    → Shard 2
Users 2M-3M    → Shard 3
```

Diagram



Sharding Solves

- ✅ write scaling
- ✅ huge data
- ✅ storage scaling

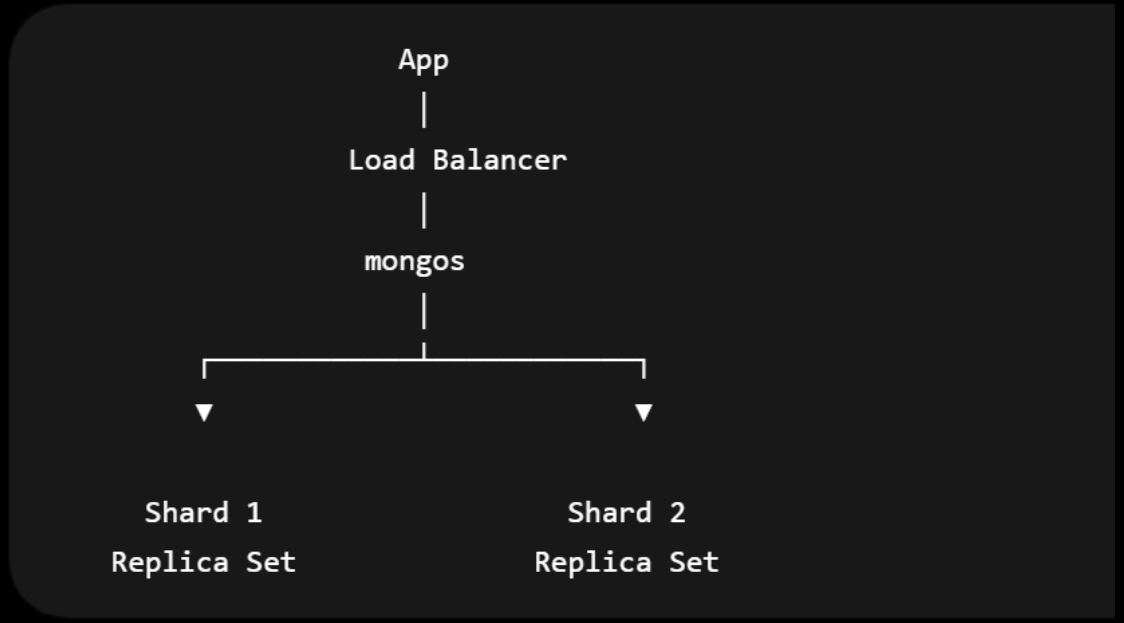
Replication vs Sharding

Feature	Replication	Sharding
Purpose	Read scaling	Write/data scaling
Data	Same copy	Divided
Storage	Repeated	Distributed
Complexity	Easier	Harder

Real Production Setup

Companies dono combine karti hain.

Architecture



3 System design

> Replication increases availability and read performance.
> Sharding increases storage and write scalability.

VIRTUAL CODE

Advanced BACKEND + AI Full Course

PART 2













In This Course

VIRTUAL CODE

Part 1

Level 0– Basic Backend Revision

Level 1 – Docker
- introduction
- Images & Containers
- How to Dockerize a node Application
- Port Mapping
- Docker Compose
- Docker Networking
- Docker Volumes

Level 2 – Redis
- introduction
- API Caching
- Rate Limiting
- Message Queue

Level 3 – System Design
- introduction
- Scaling Concepts
- Nginx
- Microservices
- Database Scaling (Replication , sharding)

Part 2

Level 4 – AI Full Course
- LLM
- Langchain
- LangGraph
- RAG
- Vector DB

Level 5 – Cloud Deployment
- AWS
- CI/CD Pipeline

Project

“Pay what you think this course deserves” 🤍



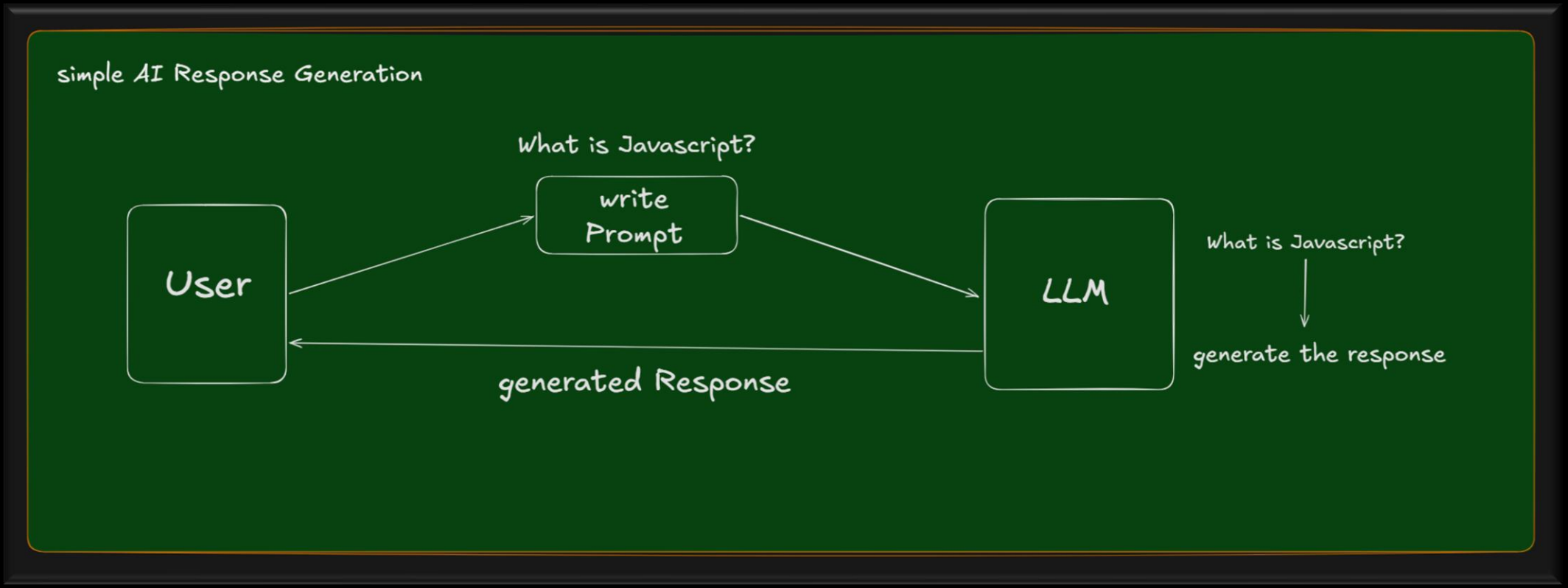
Agar aapko ye course valuable laga ho, to aap apni marzi se support kar sakte ho 🤍
Amount completely aapke upar hai — ₹1 se lekar jitna aapko sahi lage utna.
Bas agar course ne genuinely help kiya ho, to at least ₹50 support kar dena 🙌
Isse aur free & quality content banane me help milegi.

Scan QR or Support via Link in Description

VIRTUAL CODE

LEVEL 4

AI Full Course


simple AI Response Generation
What is Javascript?
write Prompt
User
LLM
What is Javascript?
generate the response
generated Response

What is an LLM?

LLM stands for **Large Language Model**.

It is an AI model trained on massive amounts of text data to understand and generate human language.

Examples:

- OpenAI GPT models
- Anthropic Claude
- Google Gemini
- Meta Llama

Simple Example

You ask:

> "Write a Python function for sorting."

The LLM predicts the next tokens and generates working code.

Important Point

An LLM by itself:

- does **not** access tools automatically
- does **not** remember long-term state reliably
- does **not** take actions in the real world unless connected to systems

It is mainly a **text intelligence engine**.

What is an AI Agent?

An AI Agent is a system that uses an LLM **plus tools, memory, and decision-making** to perform tasks autonomously.

Think of it like:

```
LLM = Brain
Agent = Brain + Tools + Actions + Memory
```

Agent Components

An agent usually has:

- LLM (reasoning)
- Memory
- Tool calling
- Planning
- Execution loop
- APIs/databases access

Example Agent Workflow

User:

> "Find the cheapest flight to Delhi and email me details."

Agent:

1. Understands request using LLM
2. Searches flights
3. Compares prices
4. Chooses best option
5. Sends email
6. Returns result

That entire workflow = AI Agent.

Difference Between LLM and Agent

Feature	LLM	Agent
Generates text	✅	✅
Uses tools/APIs	❌	✅
Makes decisions	Limited	✅
Multi-step tasks	Weak	Strong
Memory	Minimal	Persistent
Autonomous actions	❌	✅

Popular Agent Frameworks

Developers commonly use:

- LangChain ↗
- LangGraph ↗
- LlamaIndex ↗
- CrewAI ↗
- AutoGen ↗

What is LangChain?

LangChain is a framework used to build:

- AI chatbots
- RAG systems
- AI agents
- Tool-calling apps
- Multi-step AI workflows

It helps connect:

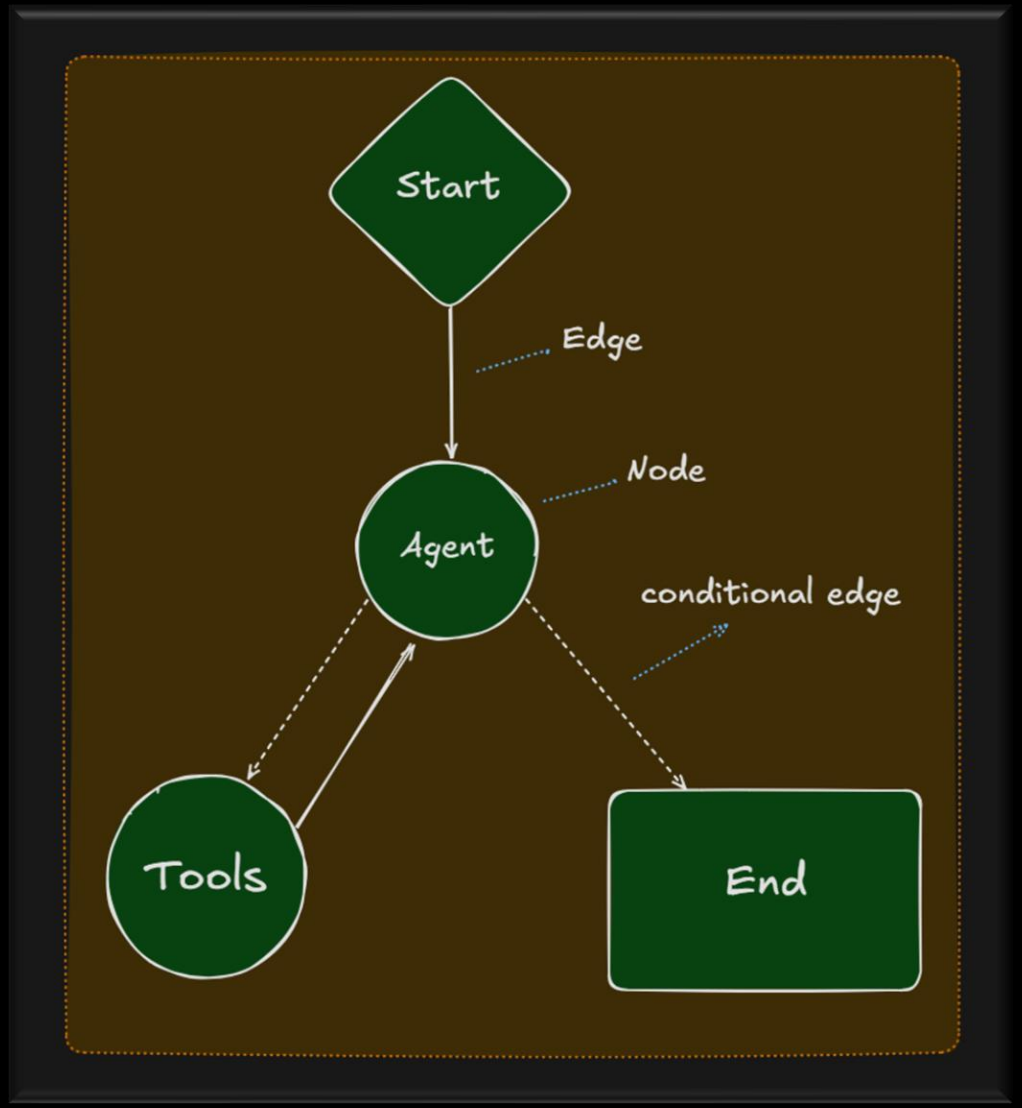
```
LLM + Memory + Tools + Vector DB + APIs
```

LangGraph

LangGraph is used for:

- Stateful AI agents
- Memory
- Multi-agent systems
- Workflow orchestration

This is production-level AI engineering.


Start
Edge
Node
Agent
conditional edge
Tools
End

RAG Kya Hota Hai?

RAG ka full form:

```
Retrieval Augmented Generation
```

Simple language me:

```
LLM + External Knowledge
```

Matlab AI sirf apni training knowledge use nahi karta.

Wo:

1. Pehle data search karta
2. Relevant information nikalta
3. Fir answer generate karta

Problem Without RAG

Suppose GPT/Gemini ko tumhara PDF nahi pata.

Tum pucho:

```
Mere project ka title kya hai?
```

Normal LLM:

```
Mujhe nahi pata
```

Because:

- model ne tumhara PDF kabhi dekha hi nahi

RAG Kya Karta

RAG:

```
PDF read karta
↓
Store karta
↓
Question aane pe relevant data fetch karta
↓
LLM ko deta
↓
LLM answer deta
```

RAG Real Flow

```
PDF
 ↓
Chunking
 ↓
Embeddings
 ↓
Vector Database
 ↓

User Question
 ↓
Similarity Search
 ↓
Relevant Chunks
 ↓
LLM
 ↓
Answer
```

Embedding Kya Hota Hai?

Embedding means:

```
Text → Numbers (Vectors)
```

Example:

```
"car booking app"

↓

[0.21, -0.82, 0.55, ...]
```

AI text ko mathematically represent karta.

Why Embeddings Needed?

Computer text samajhta nahi.

Computer:

- numbers samajhta
- vectors samajhta

So embedding model:

- text ko semantic numbers me convert karta

Semantic Meaning

Example:

```
"car"
"vehicle"
"automobile"
```

Words different hain.

Meaning similar hai.

Embedding vectors bhi close honge.

Isi wajah se semantic search possible hota hai.

Semantic Meaning

Example:

```
"car"
"vehicle"
"automobile"
```

Words different hain.

Meaning similar hai.

Embedding vectors bhi close honge.

Isi wajah se semantic search possible hota hai.

Vector Kya Hota Hai?

Vector basically:

```
numbers ki list
```

Example:

```
[0.12, 0.91, -0.44, 0.66]
```

Real embeddings:

```
768 dimensions
1536 dimensions
3072 dimensions
```

tak ke hote.

Embedding Model Kya Hota?

Embedding model:

```
text ko vector me convert karta
```

Examples:

Model	Company
text-embedding-004	Google
text-embedding-3-small	OpenAI
bge-large	HuggingFace
e5-large	Microsoft

Vector Database Kya Hota?

Normal DB:

```
exact match
```

search karta.

Example:

```
SELECT * WHERE name="car"
```

Vector DB:

```
similar meaning
```

search karta.

Example:

```
"vehicle booking"
```

search karne pe:

- car booking
- auto booking
- cab system

sab mil sakta.

Vector Database Ka Kaam

Store:

```
vector embeddings
```

Search:

```
nearest semantic vectors
```

Normal DB vs Vector DB

Normal DB	Vector DB
exact text	semantic meaning
SQL search	similarity search
structured data	embeddings
fast filters	AI retrieval

Similarity Search Kya Hota?

Suppose query:

```
What technologies are used?
```

Embedding model vector banayega.

Fir Vector DB:

- similar vectors dhundega
- relevant chunks return karega

One-Line Summary

RAG

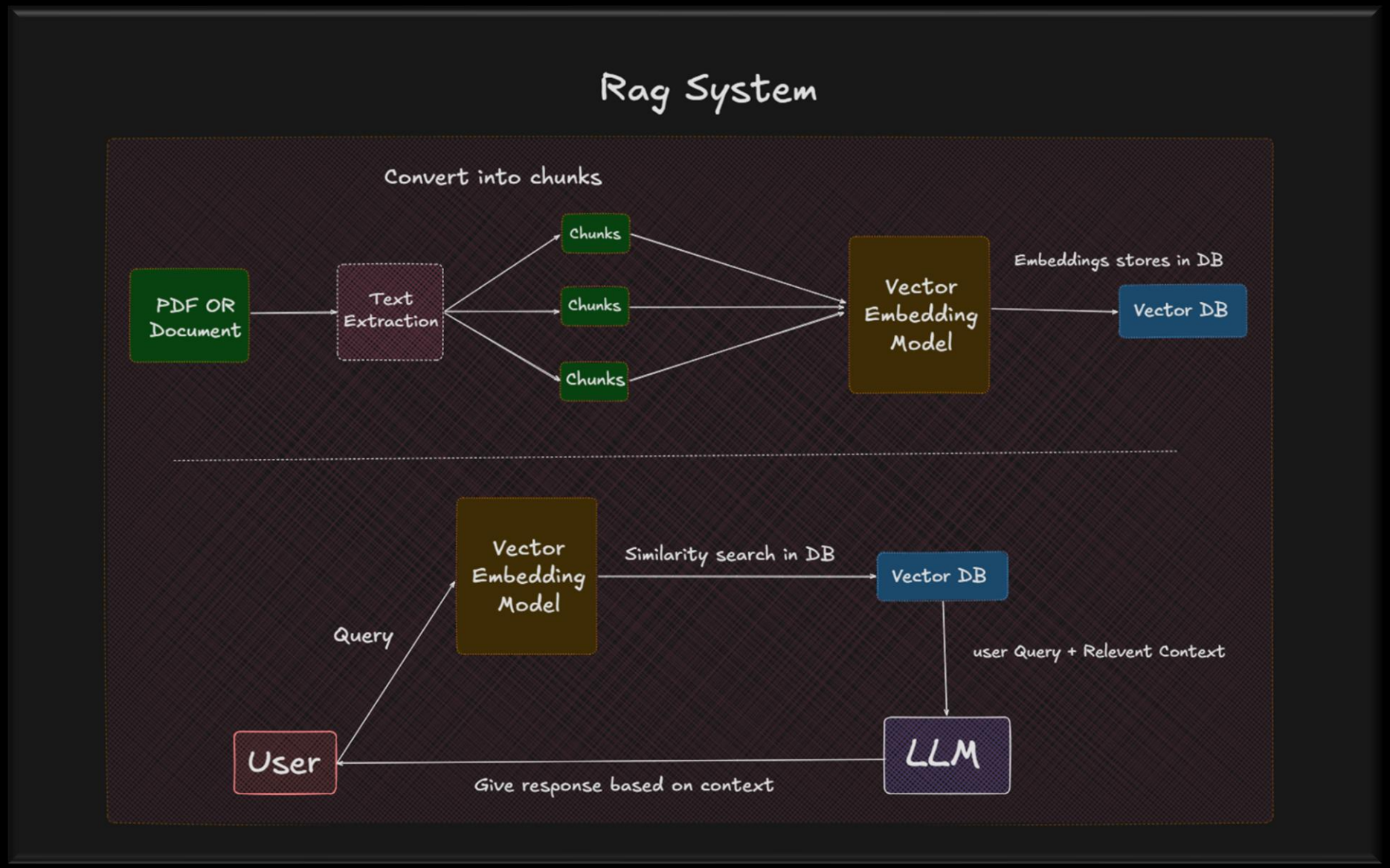
```
AI + External Knowledge Retrieval
```

Embedding

```
Text converted into semantic vectors
```

Vector Database

```
Database optimized for semantic similarity search
```


Rag System
Convert into chunks
PDF OR Document
Text Extraction
Chunks
Chunks
Chunks
Vector Embedding Model
Embeddings stores in DB
Vector DB
Vector Embedding Model
Similarity search in DB
Vector DB
Query
user Query + Relevent Context
User
LLM
Give response based on context

LEVEL 5

Cloud Deployment

5 | Cloud Deployment

1. Cloud Kya Hota Hai

Cloud basically kisi aur company ke powerful servers hote hain jo internet par available hote hain.

Instead of:

- apna physical server kharidna
- electricity
- cooling
- maintenance
- networking

Tum rent par server use karte ho.

Major cloud providers:

- Amazon Web Services
- Google Cloud
- Microsoft Azure

AWS Introduction

AWS Kya Hai

Amazon Web Services duniya ka sabse popular cloud provider hai.

AWS services bahut saari hoti hain, but deployment ke liye initially sirf kuch important services samjho.

---

1. CI/CD Kya Hota Hai

CI

Continuous Integration

Meaning:

```
code continuously integrate karna
```

Developer:

- code push kare
- automatically:
  - build
  - test
  - validation

run ho.

5 | Cloud Deployment

CD

Continuous Deployment / Delivery

Meaning:

```
automatic deployment
```

After successful build:

- server update
- docker rebuild
- ECS deploy

Automatically ho jaye.

5 | Cloud Deployment

Traditional deployment:

```
ssh server
git pull
npm install
restart app
```

Problems:

- downtime
- mistakes
- forgot restart
- wrong branch
- inconsistent deployments

CI/CD Solves

✅ Automation
✅ Faster deployment
✅ Reliable deployment
✅ Team collaboration
✅ Rollback support
✅ Production consistency

CI/CD Folder

Most important:

```
.github/workflows/
```

Inside:

```
deploy.yml
```

5 | Cloud Deployment


```yaml
name: Deploy To EC2

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Connect EC2 and Deploy
        uses: appleboy/ssh-action@v1.0.3

        with:
          host: ${{ secrets.HOST }}
          username: ubuntu
          key: ${{ secrets.SSH_KEY }}

          script: |
            cd /home/ubuntu/project

            git pull origin main

            docker compose down

            docker compose up --build -d
```